

Slide · every page may have a deck, and the deck is authored

1 One path per page, and no list to keep in step

One path per page: if the file is there, the page has a deck.

```
<page>/slide/<page>-deck.html
worked out from the page's own path, the way Draw finds its owner
no list, no registry: ask for the path and you know
```

This part settles where a deck lives, and how both doors find it with no list to keep. A folded page (`<name>/<name>.md`) owns the `slide/` folder. The deck's file name is the page name plus `-deck.html`. Both doors below work this path out from the page they are on, so no list is kept and no list can go stale. When the file is not there, both views say so and offer the button. Nothing builds a deck behind your back.

2 Two ways in, and neither can show you an old deck

Where a deck opens: the shell tab and the page panel, with one loader behind both.

```
the shell's  Slides tab      beside Chat and Draw, with the
                             bar sitting above the deck
the page's   panel          a right side panel you can resize,
                             with regenerate in its header
?plain                          ask for the deck this way, or it
                             comes back wearing the whole shell
```

This part settles that both doors read the same saved file, and that neither can show you an old one. Both doors send a HEAD request for that path, which is a small web question that asks only whether a file is there. They show the saved deck, or a pointer to the button when there is none. Every load skips the browser cache, so a freshly written deck appears on the next click. An old copy in the window can never outlive the file it came from.

3 One click rewrites the deck, and always replaces it

What the button does: the steps it runs, and the file it replaces.

```
click --> POST /_board/autodeck {file, prompt?}
--> live/autodeck.py runs `claude -p` on the page's .md
--> slide/<page>-deck.html replaced --> the view reloads
a few minutes . the status word says so while it thinks
```

This part settles that a rebuild always replaces the whole deck, so a lasting fix belongs on the page. The ask box is optional. Leave it empty and the deck presents the page's argument, or type an ask and the talk leans the way you asked. The server writes the `<head>` and its asset links itself, from real paths on disk. So the model starts writing at `<body>`, and a new deck can never miss its own stylesheets. The server also checks the result before it saves anything. It wants a whole

document, at least three slides, and the asset links in place. If any of that is missing the new deck is refused, and the old file stays. A rebuild always overwrites, so a lasting fix belongs on the PAGE, and the page is what every rebuild reads.

4 What you get: six to nine slides you can stand up and present

The look: html-ppt supplies the shell, and the board supplies the words.

```
academic-report theme . T cycles the short gallery
  move . F fullscreen . O overview . S presenter mode
every slide carries speaker notes, and S shows them
scripts blocked? the data-js stamp is missing, so CSS shows
  one slide picked by the url, and the deck still reads
```

This part settles what a deck looks like, and where that look comes from. A deck is six to nine slides: a cover, a closing slide, and middle slides in between. The middles are built from the page's own emoji, numbers, and quoted rulings. The deck points at the html-ppt skill's files by relative path. So a better theme reaches every deck at once, with nothing to rebuild. Nothing from the skill is copied into the board, and the board adds no CSS of its own.

5 Why Claude writes the deck, and what was dropped

The record: three ways a page could have had a deck, and the one still standing.

```
SELECTION . 260815 . JL ruled: "We will just have the AI deck"
winner      the deck Claude writes . claude -p writes it from the page's .md
loser       the reflow copy . live/deck.py . dropped
loser       the slide page-type . one page per deck . dropped
downstream  ../../board/page-plugins/haipipe-plugin-slide/
```

This part settles which of the three ways won, and why each of the other two was dropped. This way won on evidence, not on argument. Six decks were written in the hour of the ruling, and the first live `/_board/autodeck` run produced a real talk for `QF2-newcomer`. The reflow copy lost because it copied instead of choosing, so one part of the page came back as its own paragraphs in a box. It was dropped whole, endpoint and route with it, and nothing in the board turns a page into slides that way now. The slide page-type lost because it asked for a second page to keep beside the page it was about. That second page held one part of the page per slide, each with a frame inside it. It was dropped, and its example is kept whole at `_archive/QBt9-for-slide.md`, where the reason it lost stays readable.